2027 OKC Thunder Analyst Internship Project Writeup

Jack Clark
September 2026

1. Objective

The goal of this project was to predict the probability that an NBA shot attempt would be made using player-tracking and contextual data. Because performance is evaluated using log-loss, I focused on producing accurate and well-calibrated probabilities rather than simply classifying shots as makes or misses. My approach was to first understand the structure of the data, then compare increasingly flexible models under a validation framework designed to approximate performance on an unseen season.

2. Data Understanding and Feature Development

2.1. Data Integrity and Structure

I began by validating the structure of the training and testing data before building any models. The training set contained 425,719 shots with 29 columns, while the testing set contained 213,977 shots with 28 columns. I also checked data types, outcome validity, categorical levels, missing values, and whether the same transformations could be applied consistently to both datasets.

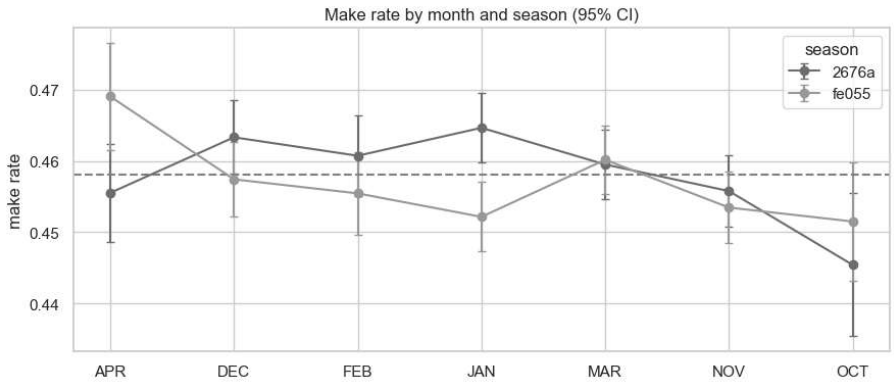
A few basic integrity checks that passed

- Training observations: 425,719
- Testing observations: 213,977
- Outcome values: 0/1 only, with no missing outcomes
- Testing data contained the same predictors as training, excluding outcome
- Missing contester fields aligned with the number of recorded contesters
- Submission template contained 213,977 rows

These checks confirmed that the datasets were structurally consistent and that most apparent missingness in the contester fields reflected the basketball situation rather than a data-quality issue.

2.2. Season and Outcome Structure

The two training seasons had very similar overall shot-make rates:
Season 2676a: 45.96%
Season fe055: 45.65%



2.3. Understanding the Tracking Variables

Several tracking variables required more interpretation before I could decide how to use them. In particular, *closestdefapproach* contains four defender-distance measurements leading up to the shot rather than a single release-time value.

Time before shot	**Mean absolute gap from release-time defender distance**
1.00 s	4.40 ft
0.75 s	3.38 ft
0.50 s	2.11 ft
0.25 s	1.13 ft

The decreasing gap showed that the approach measurements progressively converged toward the defender's release-time distance. This confirmed that the sequence captured defender movement leading into the shot and motivated features describing changes in defensive pressure over time.

2.4. Feature Development

Based on the initial analysis, I organized the predictors into basketball-relevant groups and added derived variables only when they represented information not fully captured by a single raw field.

Shot geometry: distance, court location, three-point status, shot type

Shot context: shot clock, dribbles before the shot, shooter speed, game state

Defensive pressure: number of contesters, contesters distances, closest defender distance

Defender movement: Measurements derived from *closestdefapproach*, change in defender distance leading into release

2.5. Initial Analysis Suggestions

The initial analysis suggested that shot probability was likely driven by several interacting components. Shot location and shot type described the basic difficulty of an attempt, while defensive pressure and defender movement added information about the quality of the opportunity. Because these relationships were unlikely to be purely linear, I decided to begin with a simple interpretable baseline and then test whether nonlinear models produced meaningful improvements.

3. Validation Strategy

3.1. Validation Design

Because the test set came entirely from a season not represented in training, I wanted the validation framework to reflect cross-season generalization as closely as possible. I therefore compared several splitting strategies before using any one of them for model selection.

Validation scheme	**Purpose**	**Mean log-loss**
Random 5-fold	Standard out-of-sample baseline	0.62721
Contiguous shot ID blocks	Test whether nearby observations carried hidden ordering or grouping	0.62724
Grouped by similar game	Test whether game-level similarity benefited random folds	0.62744
Grouped by shooter	Stress-test dependence on observing the same shooters in training	0.62746

Season holdout	Measure generalization to a completely different season	0.62840

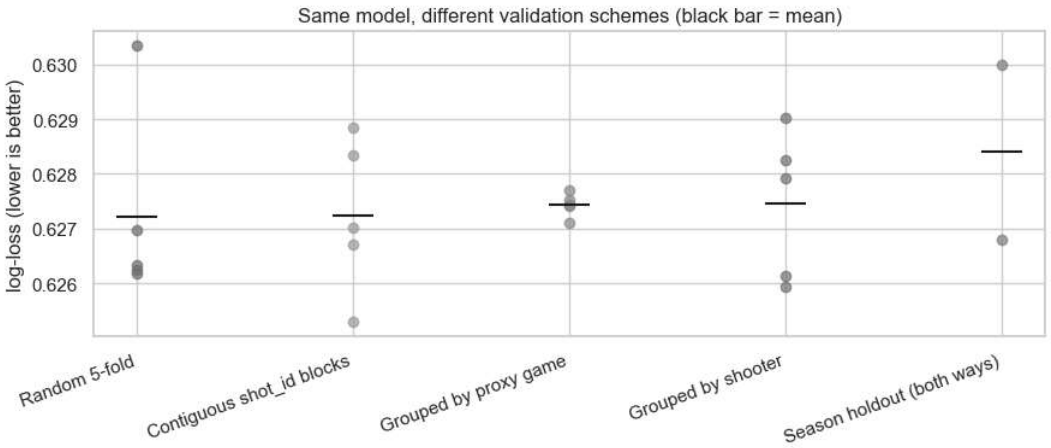


Figure: Comparison of validation log-loss across alternative splitting strategies. Season holdout was the most conservative and most closely matched the unseen-season test setting.

The first four validation schemes produced similar results, suggesting that shot ordering, game grouping, and repeated shooters were not creating large validation advantages. Season holdout was somewhat more difficult and most closely matched the final prediction setting.

For subsequent model development, I therefore used two-way season-swap validation. Models were trained on one season and evaluated on the other, then the direction was reversed. The mean of the two log-loss values was used as the primary model-selection metric.

All candidate models were evaluated using **log-loss**, matching the project metric. Because log-loss evaluates the full predicted probability and penalizes confident incorrect predictions, model selection focused on probability quality rather than classification accuracy.

3.2. Linear Baselines

I first established how much predictive information could be captured with relatively simple and interpretable models before moving to more flexible methods.

A constant model using only the training-season make rate served as the minimum baseline. I then fit a simple logistic regression using a limited set of raw predictors. An engineered logistic model added the basketball-informed features developed earlier, along with spline transformations to allow major continuous predictors to have nonlinear relationships with shot probability.

I also tested ridge and LASSO regularization to determine whether coefficient shrinkage or variable selection improved generalization.

Model	Mean season-swap log-loss
Constant make rate	0.68964
Simple logistic regression	0.64506
Engineered logistic + splines	0.63294
Ridge, C = 0.001	0.63349
Ridge, C = 0.01	0.63307
Ridge, C = 0.10	0.63299
LASSO, C = 0.001	0.63644
LASSO, C = 0.003	0.63407
LASSO, C = 0.01	0.63322

The improvement from the constant model to logistic regression confirmed substantial predictive information in the supplied features. More importantly, the engineered model improved from 0.64506 to 0.63294, indicating that domain-informed transformations and nonlinear effects contributed materially.

Neither ridge nor LASSO improved meaningfully on the unpenalized engineered model. This suggested that the next gains were more likely to come from modeling interactions and nonlinear structure than from further coefficient regularization.

3.3. Gradient Boosting and Feature-Group Development

The exploratory analysis showed likely interactions between shot geometry, shot context, and defensive pressure. I therefore moved to gradient-boosted decision trees, which can capture nonlinear relationships and interactions without requiring

each one to be specified manually.

Rather than immediately fitting the full feature set, I added groups of variables progressively.

Feature set	**Mean log**
Geometry only	0.64396
Geometry + shot context	0.63946
Geometry + context + defense	0.62802

Adding shot context improved the geometry-only model by approximately 0.0045 log-loss. The much larger improvement after adding defensive information showed that defender positioning and contest structure contained substantial information beyond shot location and shot type.
I then reversed the analysis by removing major groups from the full model.

Feature group removed	**Mean log-loss**
None, full model	0.62802
Geometry	0.65198
Context	0.63000
Defense	0.63946

Geometry was foundational to the model, while defensive information produced the largest incremental gain once geometry and context were already included.

3.4. Detailed Feature Ablation

I next performed more targeted leave-one-group-out tests to determine which portions of the engineered representation provided unique predictive value.

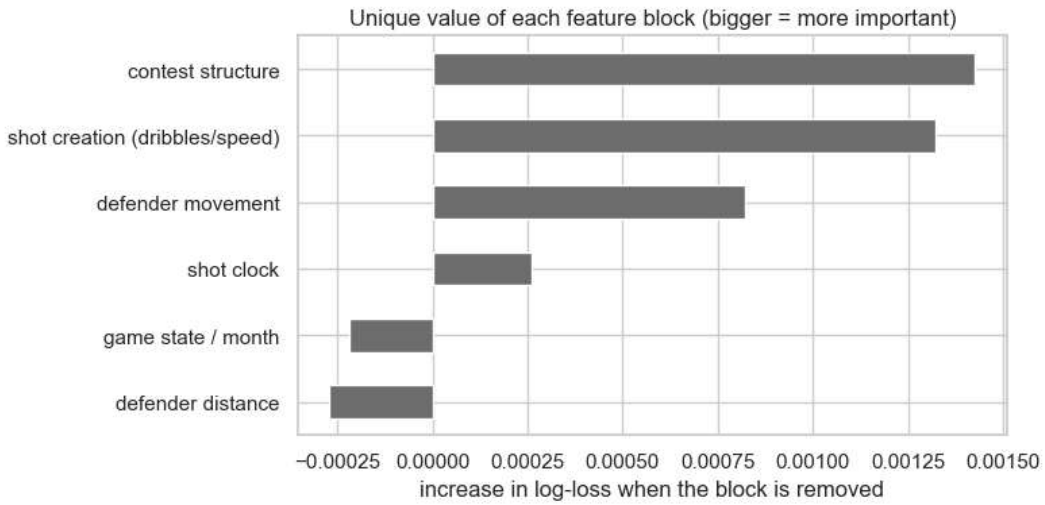


Figure: Change in season-swap log-loss after removing individual feature blocks. Positive values indicate that removing the block worsened performance, while negative values indicate little or no incremental value.

Contest structure and shot-creation features produced the largest deterioration when removed, indicating that they contributed the most unique predictive value among the tested blocks. Defender movement also added measurable signal, while standalone defender distance and game state/month provided little incremental value once the richer feature set was included.

To distinguish meaningful gains from small numerical fluctuations, I compared per-shot log-loss differences between models. Removing defender movement, contest structure, or shot-clock information each produced statistically detectable deterioration in the season-swap predictions. The tuned boosted model also significantly outperformed the engineered logistic model.

This analysis helped retain the richer basketball-based representation rather than selecting features solely from marginal correlations.

3.5. LightGBM Tuning

After establishing the feature set, I tested a small predefined set of LightGBM configurations varying tree complexity, minimum leaf size, L2 regularization, learning rate, and feature sampling.

Configuration	**Mean log-loss**

63 leaves, min leaf 300, L2 = 50, lr = .03	0.62641
31 leaves, min leaf 200	0.62641
Default 63 leaves	0.62644
15 leaves, min leaf 300, lr = .03, feature fraction = .6	0.62651
31 leaves, min leaf 500, L2 = 30	0.62659
15 leaves, min leaf 200	0.62660

The tuning range produced relatively small differences, but the best configuration reduced mean log-loss from 0.62802 to approximately **0.62641.** The best-performing specification used 63 leaves, a minimum of 300 observations per leaf, L2 regularization of 50, and a learning rate of 0.03. The relatively small spread among configurations suggested that performance was more sensitive to the feature representation than to fine hyperparameter adjustments.

3.6. Testing Persistent Player Effects

The supplied data contained high-cardinality shooter and defender IDs. Including these directly could allow the model to memorize noisy player-level outcomes, so I first tested whether player performance differences were persistent across seasons.

I fit a context-only model and calculated out-of-fold residuals for each shot. These residuals represented performance above or below what would be expected from the observed shot context.

For shooters, season-to-season residual performance showed a Pearson correlation of 0.528 across 308 players. Closest defenders showed a smaller but still positive correlation of 0.351 across 321 players.

Player effect	**Cross-season Pearson correlation**	**Estimated shrinkage constant**
Shooter	0.528	246 shots

Closest defender	0.351	524 shots

The estimated shrinkage constants were used to pull effects from players with small sample sizes toward zero, reducing the influence of noisy individual estimates.
I also tested whether shooter skill should be represented separately for three-point and non-three-point attempts. Same-type shooting performance was more persistent across seasons than cross-type performance:

Comparison	**Correlation**
3PT, Season A vs. Season B	0.360
Non-3PT, Season A vs. Season B	0.436
3PT vs. non-3PT	0.208
Non-3PT vs. 3PT	0.163

This suggested that shooting performance is not completely one-dimensional, although I retained a parsimonious shrunk player-effect representation rather than introducing increasingly sparse player-by-shot-type effects.

For defenders, I also removed team-level defensive effects before measuring persistence. The cross-season defender correlation remained essentially unchanged, increasing from 0.351 to 0.356. This indicated that the defender signal was not simply capturing team defensive quality.

Leakage control

Player effects required additional care because they were derived from outcomes. For training observations, shooter and defender effects were therefore cross-fitted: each row received a player effect estimated without using that row's fold. Validation observations received effects estimated only from the corresponding training season.

This prevented a shot's own outcome from contributing to its player-effect feature.

Model	**Season 1 holdout**	**Season 2 holdout**	**Mean**

Tuned LightGBM	0.62822	0.62460	0.62641
+ shrunk player effects	0.62671	0.62286	0.62479

Adding player effects improved mean log-loss by approximately 0.00162. The paired 95% confidence interval for the change was [-0.00187, -0.00138], supporting the conclusion that the player information added signal beyond the contextual shot features.

This improvement was larger than the gain from LightGBM hyperparameter tuning itself.

3.7. Alternative Boosting Implementations and Player Representations

After developing the LightGBM model, I tested whether the result depended on the boosting implementation or on the feature representation.

CatBoost was useful for testing whether its native categorical treatment of player and team IDs could outperform the manually estimated player effects. XGBoost provided a second comparison using another gradient-boosting implementation with native categorical support.
Each algorithm was tested in two ways:

1) Using raw player and team IDs and relying on the algorithm's native categorical handling.
2) Using the same engineered feature set and shrunk player effects developed for LightGBM.

Model	**Mean log-loss**
LightGBM + shrunk player effects	0.62479
XGBoost + engineered features + shrunk effects	0.62480
CatBoost + engineered features + shrunk effects	0.62487
CatBoost + raw player/team IDs	0.62705
XGBoost + raw player/team IDs	0.63984

LightGBM, CatBoost, and XGBoost were effectively tied when given the same engineered representation. In contrast, both approaches based primarily on raw high-cardinality IDs performed worse.

The feature representation was now confirmed to matter more than the specific gradient-boosting library. The manually estimated and shrunk player effects also generalized better than directly exposing raw IDs to the tree models.

3.8. Neural Network with Player Embeddings

I also tested a neural network to evaluate whether learned embeddings could capture player and team information more effectively than the shrunk effects used in the tree models.

The network used embeddings for high-cardinality categorical variables together with the engineered continuous features. This approach allowed the model to learn multidimensional representations of players and teams rather than reducing each player to a single scalar effect.

The neural network reached a mean season-swap log-loss of 0.63266, which was materially worse than the boosted-tree models at approximately 0.6248. The paired difference relative to the boosted model was approximately +0.00678, with a 95% confidence interval of [+0.00633, +0.00723].

Although the neural network did not perform well enough to serve as a standalone model, its predictions were structurally different from those of the boosted trees. I therefore carried it forward only as a possible ensemble component.

3.9. Model Development Summary

The model-development process produced several consistent findings.
- Nonlinear models substantially outperformed linear baselines.
- Geometry established the baseline difficulty of a shot, while defensive information added substantial predictive value.
- Contest structure, shot creation, and defender movement each provided incremental signal.
- Hyperparameter tuning improved performance, but less than feature development.

- Shooter and defender effects were persistent across seasons and improved prediction after shrinkage and cross-fitting.
- LightGBM, XGBoost, and CatBoost achieved nearly identical performance when given the same engineered feature representation.
- Raw player IDs and learned neural embeddings did not outperform the shrunk player-effect approach.

The three strongest individual models were therefore LightGBM, XGBoost, and CatBoost with the same engineered features and shrunk player effects. Their validation scores were nearly identical, but their predictions were not perfectly correlated, so all three were carried forward into final ensemble testing.

4. Final Model Selection and Calibration

4.1. Final Candidate Comparison

The strongest individual models were LightGBM, XGBoost, and CatBoost using the same engineered basketball features and cross-fitted shrunk shooter and defender effects.

Model	**Mean season-swap log-loss**
LightGBM	0.62479
XGBoost	0.62480
CatBoost	0.62487

The three models were effectively tied. Since they used the same feature representation but different boosting implementations, I tested whether averaging their probabilities could reduce model-specific error.

4.2. Ensemble Selection

I next compared the three individual boosted models with an ensemble formed by averaging their predicted probabilities.

The optimized ensemble weights converged to approximately equal weights across all validation splits:

LightGBM = 0.333
CatBoost = 0.333
XGBoost = 0.333

Because fitted weights did not improve on a simple average, I selected an equal-weight ensemble.

$$\hat{p}_{ensemble} = \frac{\hat{p}_{LGBM} + \hat{p}_{CatBoost} + \hat{p}_{XGBoost}}{3}$$

Model	**Mean log-loss**
Best individual model within each split	0.62474
Equal-weight 3-model ensemble	0.62429

Across repeated validation splits, optimized weights consistently converged to approximately one-third for each model, so the equal-weight ensemble was selected for simplicity and stability.

4.3. Neural Network Ensemble Test

Although the neural network performed worse as a standalone model, I tested whether its structurally different predictions could improve the ensemble.

The four-model ensemble weights were not chosen manually. For each validation split, I optimized nonnegative model weights constrained to sum to one, selecting the combination that minimized log-loss. This process was repeated across multiple fit/evaluation splits, and the resulting weights were averaged.

Model	**Average ensemble weight**
LightGBM	32.9%
CatBoost	32.2%
XGBoost	32.4%

PyTorch	2.6%

The optimizer assigned nearly equal weight to the three boosted-tree models but only 2.6% to PyTorch, indicating that the neural network contributed little incremental predictive value. The resulting four-model log-loss remained 0.62429, effectively identical to the three-tree ensemble.
Because the neural network added complexity without improving validation performance, I excluded it from the final model.

4.4. Probability Calibration

Because the project is evaluated using log-loss, I also examined whether the predicted probabilities were well calibrated.

The current calibration analysis showed an expected calibration error of 0.0143 and a Brier score of 0.21959. I also compared two post-hoc calibration methods:

Calibration method	**Log-loss**
Raw	0.62429
Platt scaling	0.0.62429
Isotonic regression	0.62423

Isotonic regression produced a small additional improvement over the raw ensemble, reducing mean log-loss from 0.62429 to 0.62423. Because the improvement was modest, I treated calibration as a final probability refinement rather than a major source of predictive gain.

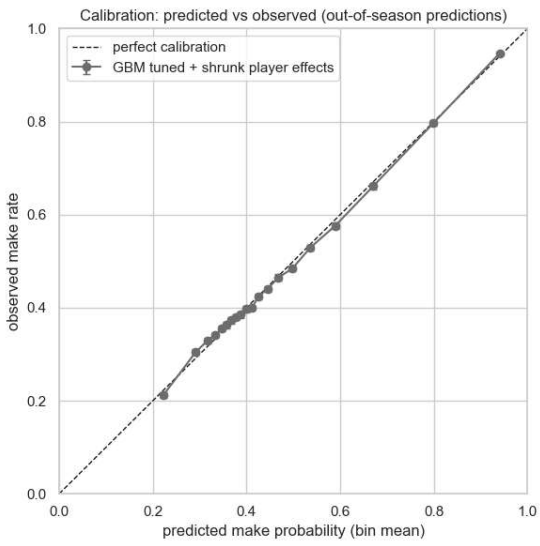


Figure: Reliability plot for out-of-season predictions from the final three-model ensemble.

4.5. Final Model Specification

Based on season-swap validation, the final predictive model was an equal-weight ensemble of LightGBM, CatBoost, and XGBoost. All three models used the same engineered representation, including shot geometry, shot context, defensive pressure, defender movement, and shrunk shooter and defender effects. The ensemble probabilities were then calibrated using isotonic regression, which provided a small additional improvement in held-out log-loss.

The raw ensemble achieved a mean season-swap log-loss of 0.62429, which improved to 0.62423 after held-out isotonic calibration.

4.6. Full-Data Training and Submission

After selecting the final specification, I retrained LightGBM, CatBoost, and XGBoost on all available training observations. Model-specific iteration counts were chosen from the median best iteration observed during validation: 311 rounds for LightGBM, 903 iterations for CatBoost, and 277 rounds for XGBoost.

The three test-set probabilities were averaged and then transformed using the selected isotonic calibrator. The final submission contained predictions for all 213,977 test shots. No probabilities were missing, and the original shot_id ordering was preserved. Shooters or defenders not observed in training received a neutral player effect of zero; these represented 0.54% and 0.76% of test shots, respectively.

5. Limitations and Future Improvements

5.1. Generalization to Future Seasons

The primary limitation is that model development was based on only two training seasons, while the final predictions are for a third unseen season. Season-swap validation was designed to approximate this setting, but two seasons provide limited information about how relationships change over longer periods.

With additional seasons, I would evaluate the model using rolling or leave-one-season-out validation. This would provide a stronger estimate of year-to-year stability and allow player effects to incorporate a longer performance history.

5.2. Player Effects

The shrunk shooter and defender effects improved validation performance, but

they summarize each player using a relatively simple historical adjustment. They may not fully capture changes in role, development, injuries, roster context, or interactions between specific players.

The model also assigns a neutral effect to players who were not observed in training. This affected only 0.54% of test shots for shooters and 0.76% for closest defenders, but additional historical data could provide better estimates for these cases.

With more time, I would explore player effects that vary by shot type or location while maintaining the same shrinkage and cross-fitting safeguards used in the current model.

5.3. Additional Tracking and Context

The supplied features describe shot geometry, defensive pressure, and several aspects of shot creation, but additional tracking information could provide a more complete description of shot difficulty.

Useful additions could include:
-shooter and defender velocity vectors rather than only scalar speed or distance
-locations of all ten players at release
-passing and possession sequences before the shot
-screen and pick-and-roll information
-remaining game time and score differential
-lineup and matchup context

These variables could help distinguish shots that appear similar in the current feature set but were created under substantially different conditions.

5.4. Rare Shot Types and Data Sparsity

The model had much less information for uncommon shot types and extreme situations. For example, heaves represented only a small portion of the data and showed noticeably larger calibration error than common shot types.

With more observations, I would examine whether rare attempts benefit from specialized features or partial pooling across related shot types rather than relying entirely on the global model.

5.5. Further Model Development

The final ensemble already showed that combining different boosting implementations could reduce model-specific error. With additional time, I would focus on improving the existing pipeline rather than simply adding more model families.
I would want to add some additional features such as:
-broader cross-season hyperparameter validation
-additional interaction features motivated by basketball context
-more granular but properly regularized player effects
-ensemble calibration using fully out-of-sample predictions
-testing whether ensemble weighting remains stable across additional seasons

6. Conclusion

This project developed a probabilistic model of NBA shot outcomes using shot geometry, offensive context, defensive pressure, defender movement, and persistent player-level effects. The final model combined LightGBM, CatBoost, and XGBoost using an equal-weight ensemble. Across two-way season-swap validation, the raw ensemble achieved a mean log-loss of 0.62429, with isotonic calibration providing a small additional improvement to 0.62423.

The modeling process showed that feature representation was more important than the choice among individual gradient-boosting implementations. Defensive information, contest structure, shot creation, and persistent shooter and defender effects all contributed meaningful predictive value. The final approach therefore combined basketball-informed feature engineering, leakage-safe player estimates, cross-season validation, model averaging, and probability calibration to produce the submitted shot-make probabilities.